DESIGN DOCUMENT

Retail Transaction Analytics Platform

End-to-end data engineering and analytics application using Excel, ETL, Parquet, MySQL, FastAPI, JWT/RBAC, and React.

Document purpose
Describe the business problem, implemented architecture, data-processing choices, technology stack, security model, validation results, and known limitations for project evaluation.

Implementation status: core analytics workflow reported working during development. This document describes the implemented analytics system and does not claim unimplemented GenAI, RAG, vector-search, or agent functionality.

1. Executive Summary

The Retail Transaction Analytics Platform processes historical online retail transaction records and makes the cleaned data available for analytics through a relational database, REST API, and browser-based dashboard. The pipeline combines two Excel worksheets, performs validation and transformation, writes a Parquet artifact, and loads the cleaned transaction data into MySQL. FastAPI exposes analytics, customer, product, and authentication routes. A React/Vite interface presents Overview, Sales Analytics, Customer Analytics, and Product Management pages.

The project is designed as a locally runnable application. It combines data engineering, backend API development, database storage, frontend visualization, and access control in one repository.

2. Problem Statement and Business Objective

Problem statement. Historical retail workbooks can contain duplicate rows, missing fields, negative quantities, zero or negative prices, and other quality issues. Directly analyzing the raw workbook makes consistent reporting and repeatable access more difficult.

Business objective. Build a repeatable processing pipeline and application that organizes retail transactions, exposes useful aggregate and record-level information, and lets authorized users explore sales, customer, and product information through a dashboard.

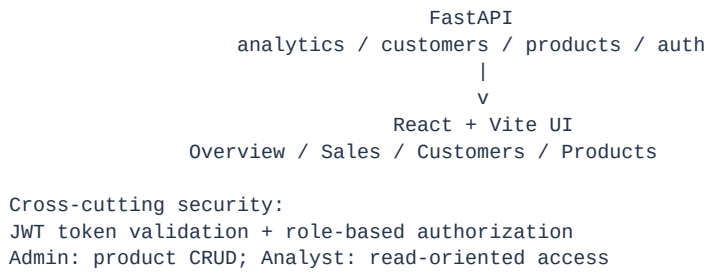
Intended users. Analysts who need read access to analytics and product data, and administrators who also need product-management capabilities.

3. Scope and Implemented Capabilities

Area	Implemented capability
Data ingestion	Read the two worksheets in Online_Retail_II.xlsx and combine records.
ETL and quality	Extract, validate, transform, clean, and report data-quality findings.
Processed artifact	Write cleaned records to data/processed/cleaned_retail.parquet.
Storage	Load cleaned records into the MySQL retail_analytics database, transactions table.
API	FastAPI route groups for authentication, analytics, customers, and products.
Security	JWT authentication and admin/analyst role-based access; product mutations restricted to admin by backend.
Frontend	React/Vite dashboard pages for Overview, Sales Analytics, Customer Analytics, and Product Management.

4. High-Level Architecture

The architecture follows a data pipeline into a serving layer. The Parquet file is a processed artifact and the MySQL table is the database source used by the API. Authentication and authorization protect applicable backend endpoints; the React UI is a client of the API.



The separate architecture diagram should be stored at architecture/Architecture_Diagram.pdf. It illustrates the same actual flow and does not add an LLM, embedding model, vector database, RAG pipeline, or agent layer.

5. ETL Design and Decisions

Extraction. Read the Year 2009-2010 and Year 2010-2011 worksheets from the source workbook and combine them into a common dataset.

Validation and profiling. Inspect data types, dates, missing fields, duplicates, quantities, and prices. The pipeline reports data-quality counts before cleaning.

Transformation. Standardize the transaction columns and derive fields such as transaction_value, is_cancelled_invoice, and is_negative_quantity. Preserve source_sheet to support traceability to the input worksheet.

Cleaning and storage. The pipeline produces a Parquet output for efficient local analytical storage and loads the cleaned records to MySQL for API access.

Decision and trade-off. Parquet provides a portable columnar processing artifact; MySQL provides a relational serving layer that supports API queries. The project retains explicit quality reporting because some anomalies—such as negative quantities and missing customer identifiers—can have business meaning and should not be interpreted blindly.

6. Technology Selection

Technology	Role and rationale
Python + pandas	Data manipulation, worksheet combination, transformation, profiling, and cleaning.
Excel reader libraries	Read the workbook sheets. Confirm exact versions from the active environment.
Apache Parquet	Store the processed dataset in a compact columnar format.
MySQL	Relational storage for cleaned transactions and SQL-based aggregation.
FastAPI	Expose backend REST endpoints and interactive OpenAPI/Swagger documentation.
SQLAlchemy + PyMySQL	Database access layer and MySQL connectivity.
React + Vite + JavaScript/CSS	Interactive web interface and frontend build tooling.
JWT + pwdlib	Token-based authentication and password hashing, as described in the README.

No LLM, embedding model, vector database, RAG framework, or agent orchestration framework is identified in the current README or verified project description. These are therefore not listed as implemented technologies.

7. Database and Data Model

The MySQL database is named **retail_analytics** and the principal transaction table is **transactions**. The processed dataset schema contains 12 columns:

Column	Purpose
invoice_no	Invoice identifier.

Column	Purpose
stock_code	Product/stock identifier.
description	Product description; may be missing in source records.
quantity	Quantity recorded for the transaction row.
invoice_date	Transaction date and time.
unit_price	Unit price recorded in the row.
customer_id	Customer identifier when available.
country	Country associated with the transaction.
source_sheet	Origin worksheet identifier.
transaction_value	Derived transaction-row value.
is_cancelled_invoice	Flag identifying cancelled invoices.
is_negative_quantity	Flag identifying negative quantities.

8. Authentication and Authorization

The application uses JWT access tokens. The admin role can manage products, while the analyst role is intended for analytics and read access. Product create/update/delete operations are enforced at the backend and should not rely solely on hiding frontend controls. The JWT signing secret and database credentials must be supplied locally and must not be committed to the repository.

9. Verification Results

Metric	Reported result
Records extracted	1,067,371
Records after cleaning / in MySQL	1,055,233
Columns in processed output	12
Unique invoices	53,623
Unique products	5,131
Unique customers	5,942
Unique countries	43
Date range	2009-12-01 07:45:00 to 2011-12-09 12:50:00
Total quantity	10,575,682
Total transaction value	19,390,476.658
Average transaction value per row	18.3755
Average order value	361.61

These values were reported from development runs and database queries. They should be presented as project verification results, not as audited financial statements or a claim of production-scale deployment.

10. Challenges, Trade-offs, and Limitations

• The source data contains missing customer IDs and descriptions, duplicate rows, negative quantities, and zero/negative prices. Cleaning rules must preserve valid cancellations and returns where appropriate.

• The difference between extracted and cleaned records is 12,138, while the data-quality report separately lists 12,133 duplicate records. The ETL rules should be inspected to reconcile the full difference; not every removed record should automatically be described as a duplicate.

• Missing customer identifiers limit customer-level analysis for affected transactions.

• The API's precise route parameters and response models should be confirmed from the running FastAPI Swagger UI at /docs.

• Local development commands and successful development checks do not prove a production deployment. Automated test results and frontend production build status should be reported only after running them.

• The current documented application is an analytics platform. A GenAI/RAG/agent workflow and its evaluation metrics are not evidenced in the supplied README; do not claim those capabilities unless implemented and tested.

11. Local Execution Overview

Create and activate a Python virtual environment, install requirements, configure local database and JWT environment settings, prepare the MySQL schema, and place the source workbook at data/raw/Online_Retail_II.xlsx. Run the ETL with python run_etl.py. Start the API with uvicorn src.api.main:app --reload, then start the frontend from frontend/ with npm install and npm run dev. Follow the repository README for details.

12. Conclusion

The project combines repeatable data preparation, a Parquet processing artifact, a MySQL data store, API-based access, role-based security, and a React analytics interface. The verified record counts and data-quality report provide evidence for the data engineering workflow. Remaining evaluation work should focus on a clean setup, repeatable tests, API documentation, and transparent reporting of any GenAI-related requirements that are not part of the current implementation.